

from Beginner to Master

DevOps를 위한 **Docker** 가상화

Section 6. Continuous Integration and Continuous Deployment

```
book:
  def setData(self, title, price, author):
    self.title = title
    self.price = price
    self.author = author

var fs = require('fs');
fs.readFile('./ONE.txt', '* 1 *',
  function (err, data) {
    console.log(data); // 3
  });
ApplicationDelegate > {
```

```
public static void main(String[] args)

<servlet>
  <servlet-name>BoardController</servlet-name>
  <servlet-class>com.joneconsulting.controller.BoardCont
  <init-param>
    <param-name>user_name</param-name>
    <param-value>henneth Lee</param-value>
  </init-param>
</servlet>
```

Winterface

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

수정하기

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명



멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명

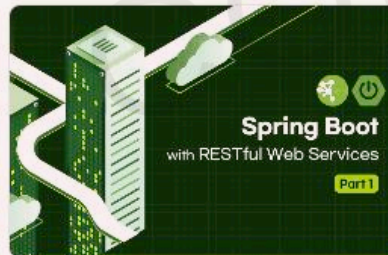


Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

이 되었던 때가 있었습니
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구
사람입니다. 개발자로서, 강
지만, 그래도, 남들보다 조

ecture, Blockchain,

- Section 0: Introduction to Cloud Native Technologies
- Section 1: Docker Essentials - Container
- Section 2: Docker Essentials - Image
- Section 3: Docker Network and Storage
- Section 4: Building and Managing Containerized Application
- Section 5: Container Orchestration
- **Section 6: Continuous Integration and Continuous Deployment**
- Section 7: Docker Security
- Section 8: Logging and Monitoring
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

Section 6.

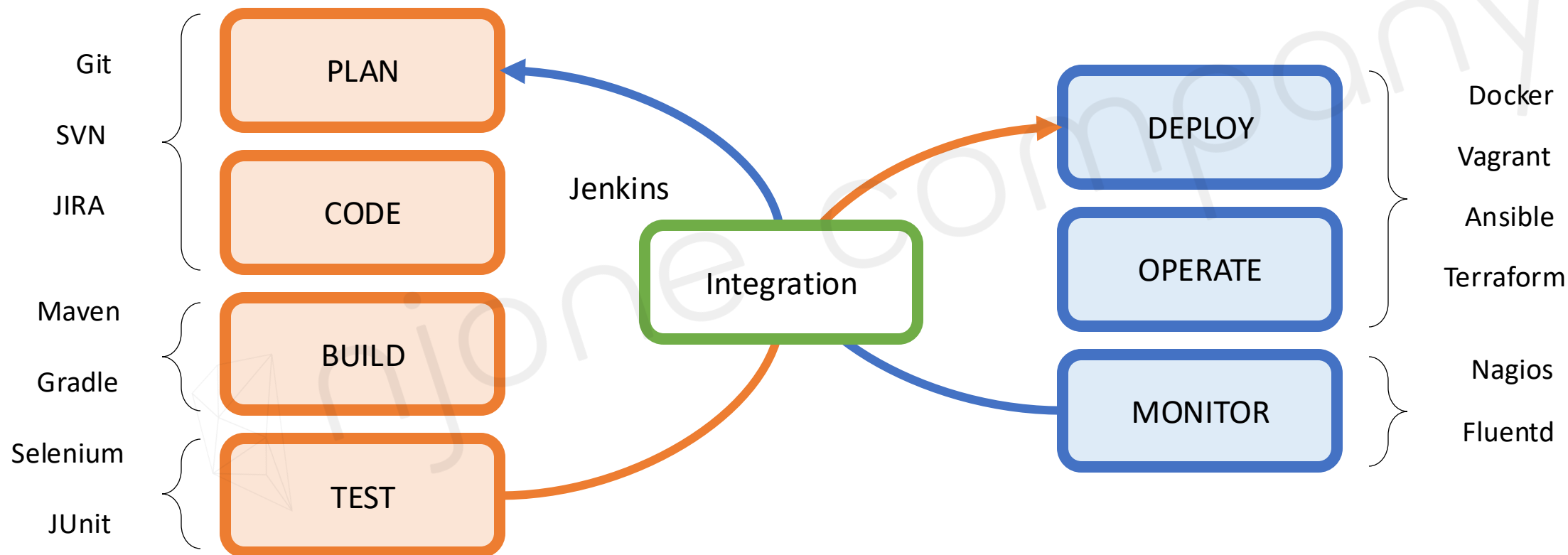
Continuous Integration and Continuous Deployment

- CI/CD 환경을 위한 Docker 컨테이너 활용
- 상용 클라우드에서의 애플리케이션 배포와 관리
- 관리형 컨테이너 서비스 사용

CI/CD 환경을 위한 Docker 컨테이너 활용

DevOps 환경에서의 Docker 활용

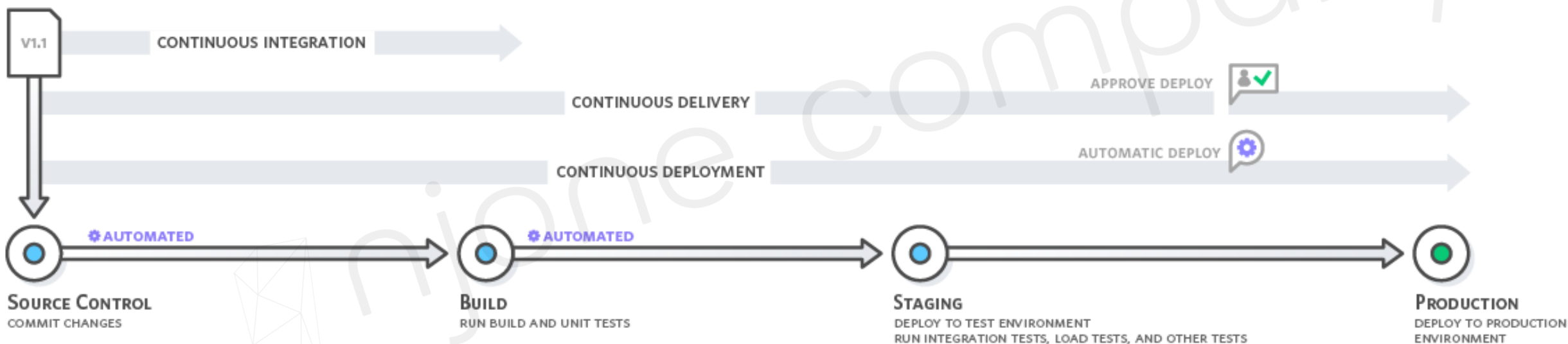
- 엔지니어가, 프로그래밍하고, 빌드하고, 직접 시스템에 배포 및 서비스를 운영
- 사용자와 끊임 없이 Interaction하면서 서비스를 개선해 나가는 일련의 과정, 문화



CI/CD 환경을 위한 Docker 컨테이너 활용

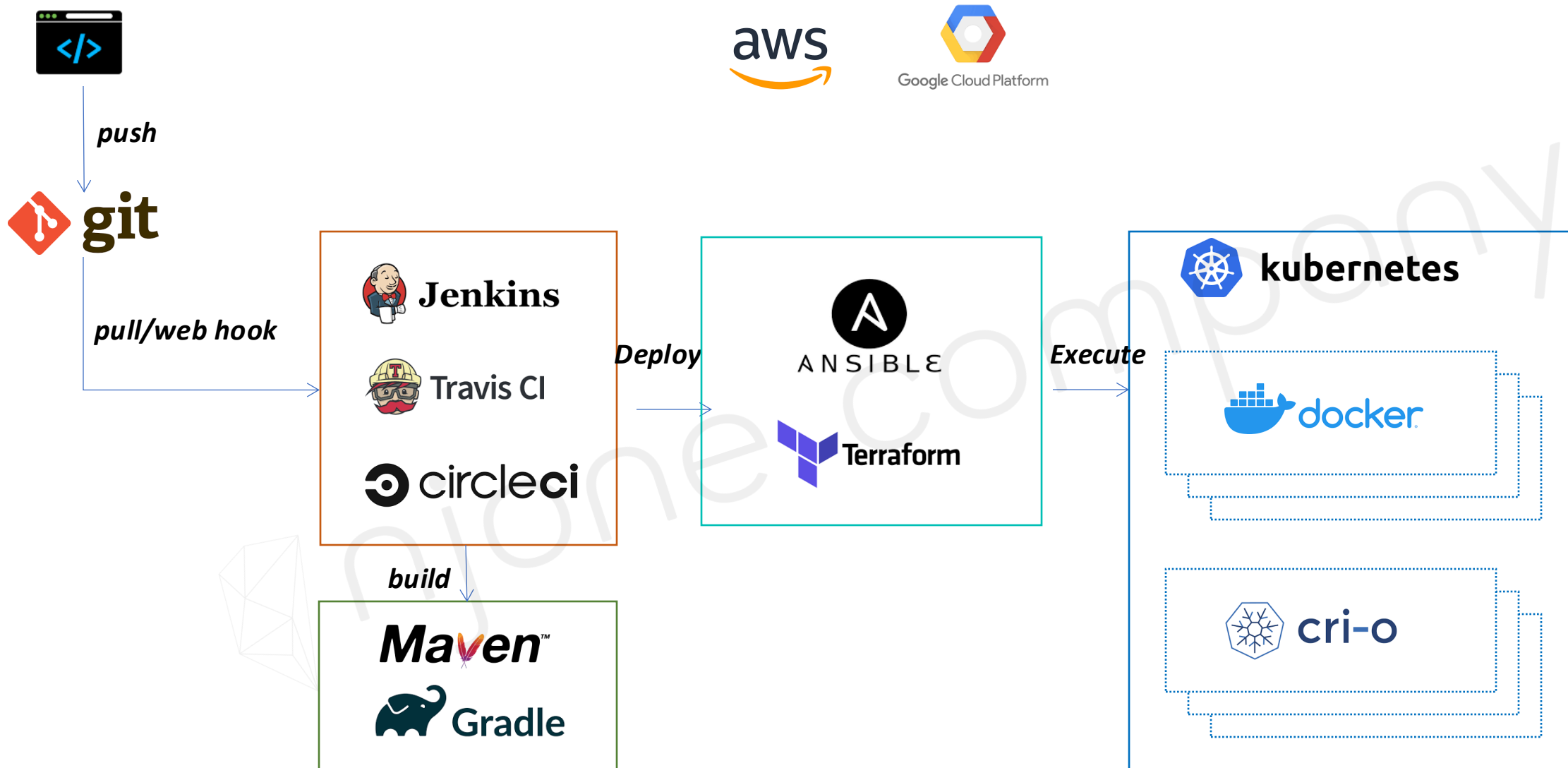
CI/CD 자동화 배포

- Continuous Integration, CI
- Continuous Delivery, CD
- Continuous Deployment, CD



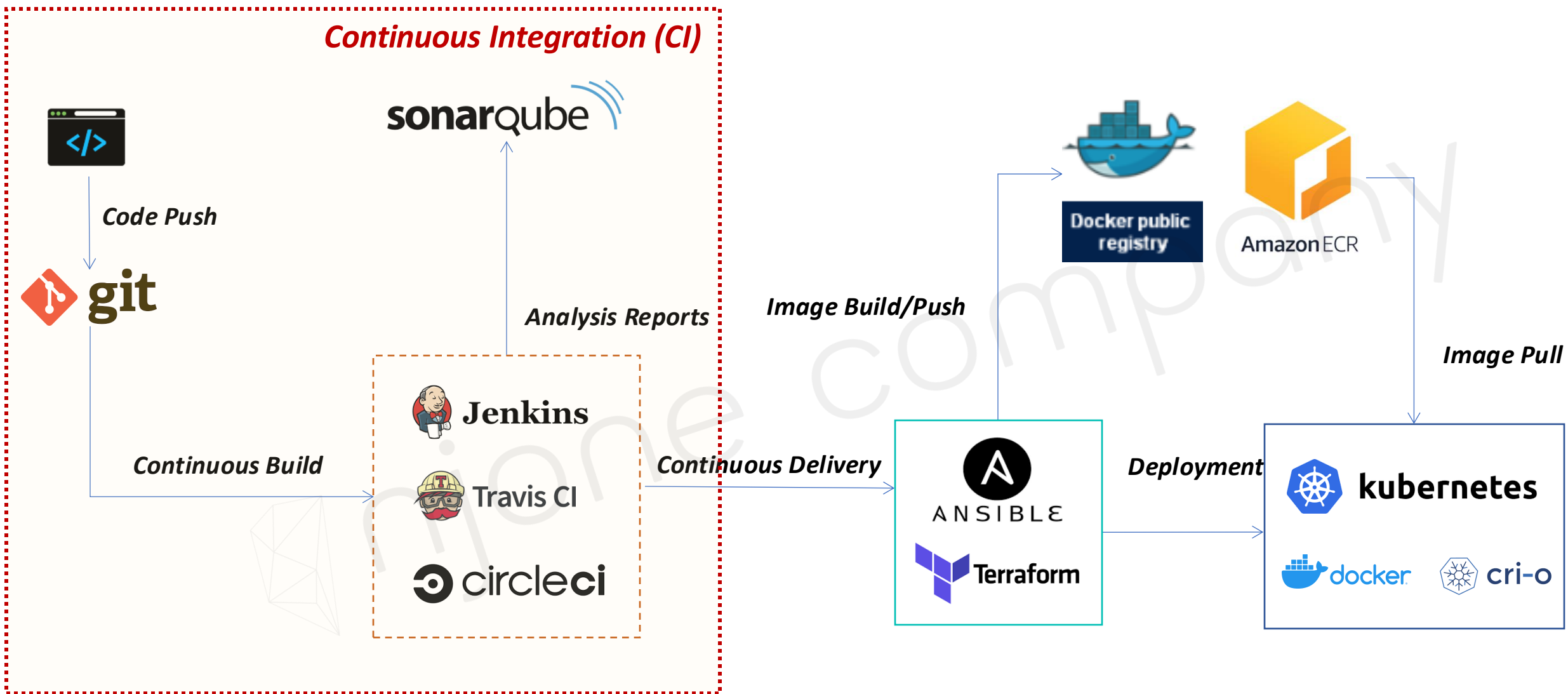
CI/CD 환경을 위한 Docker 컨테이너 활용

CI/CD 환경



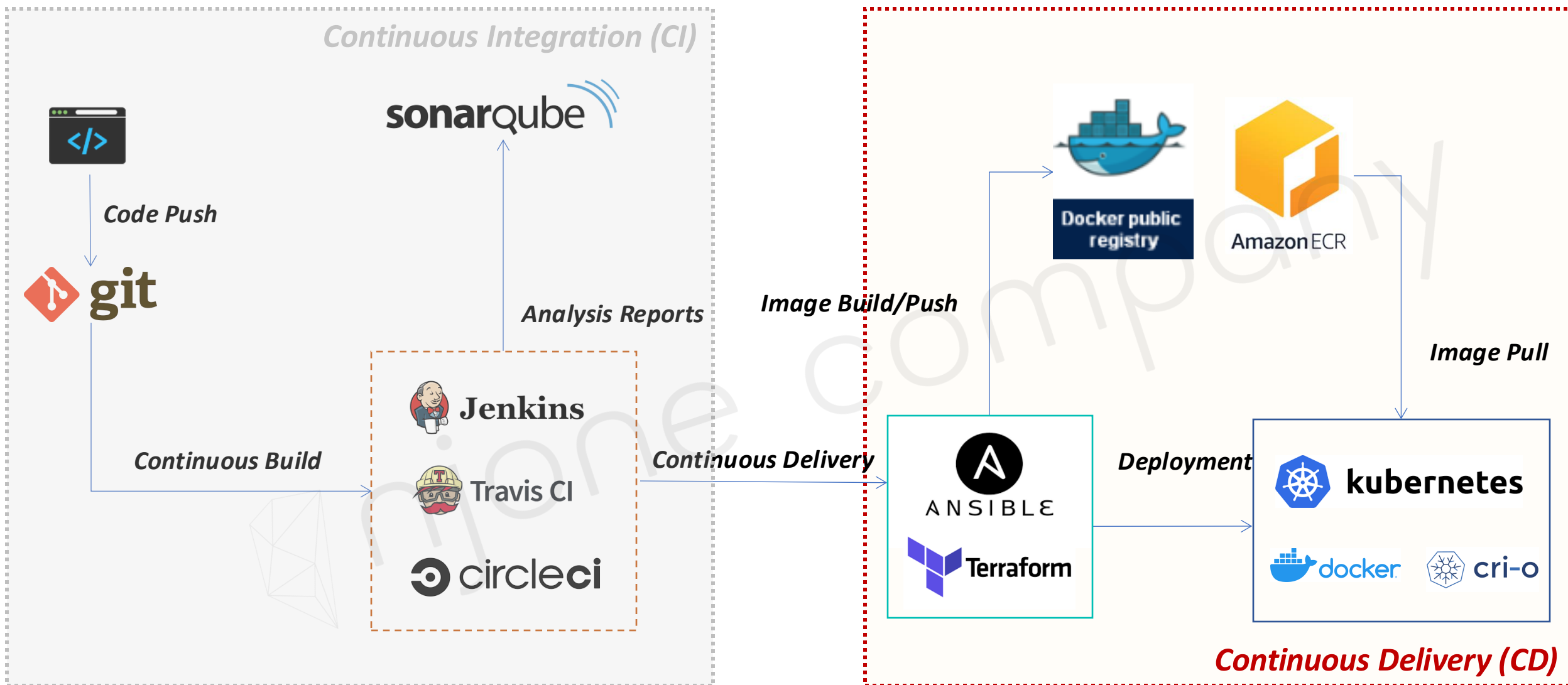
CI/CD 환경을 위한 Docker 컨테이너 활용

CI/CD 환경과 Docker



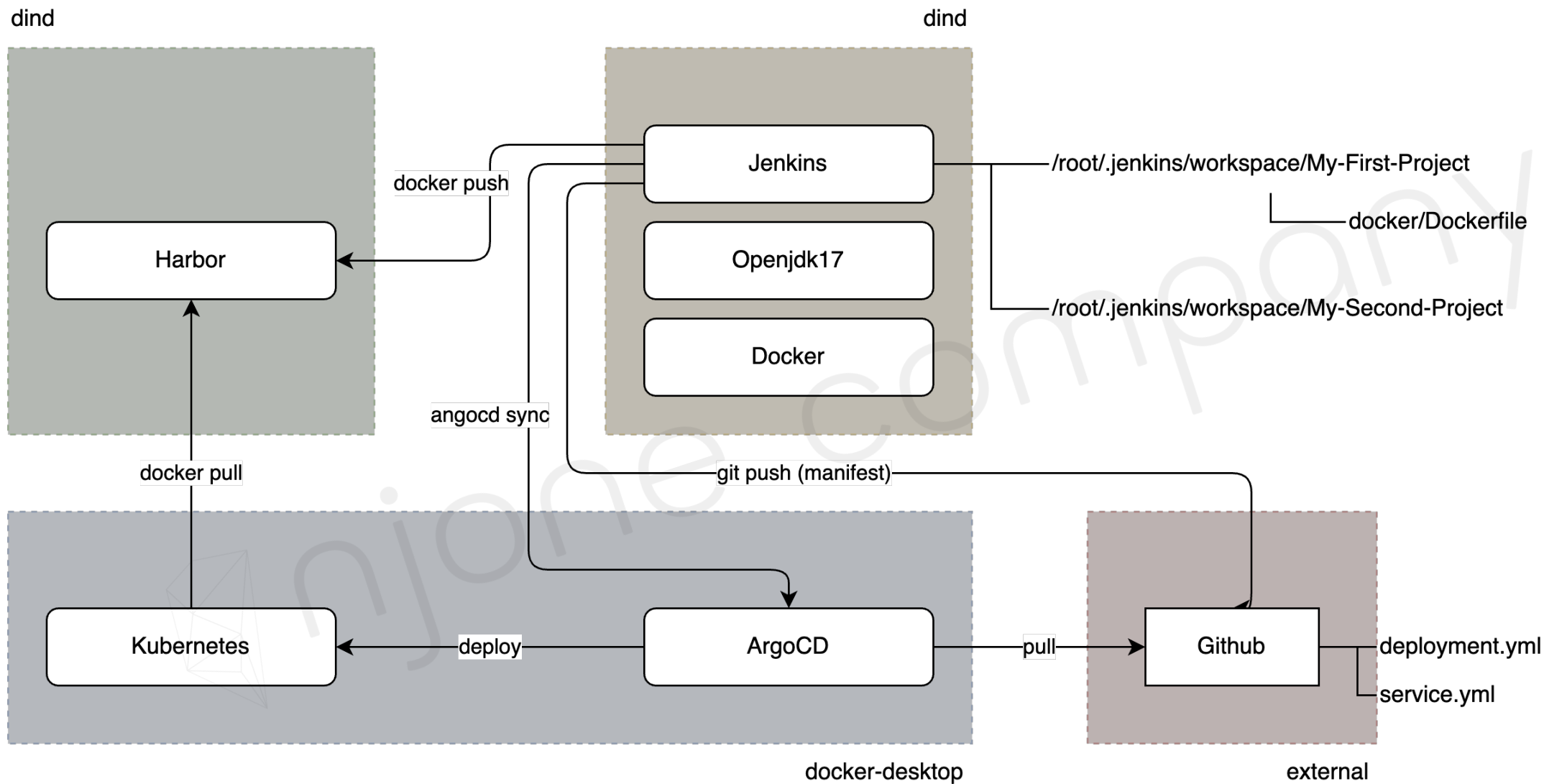
CI/CD 환경을 위한 Docker 컨테이너 활용

CI/CD 환경과 Docker



CI/CD 환경을 위한 Docker 컨테이너 활용

CI/CD Pipeline

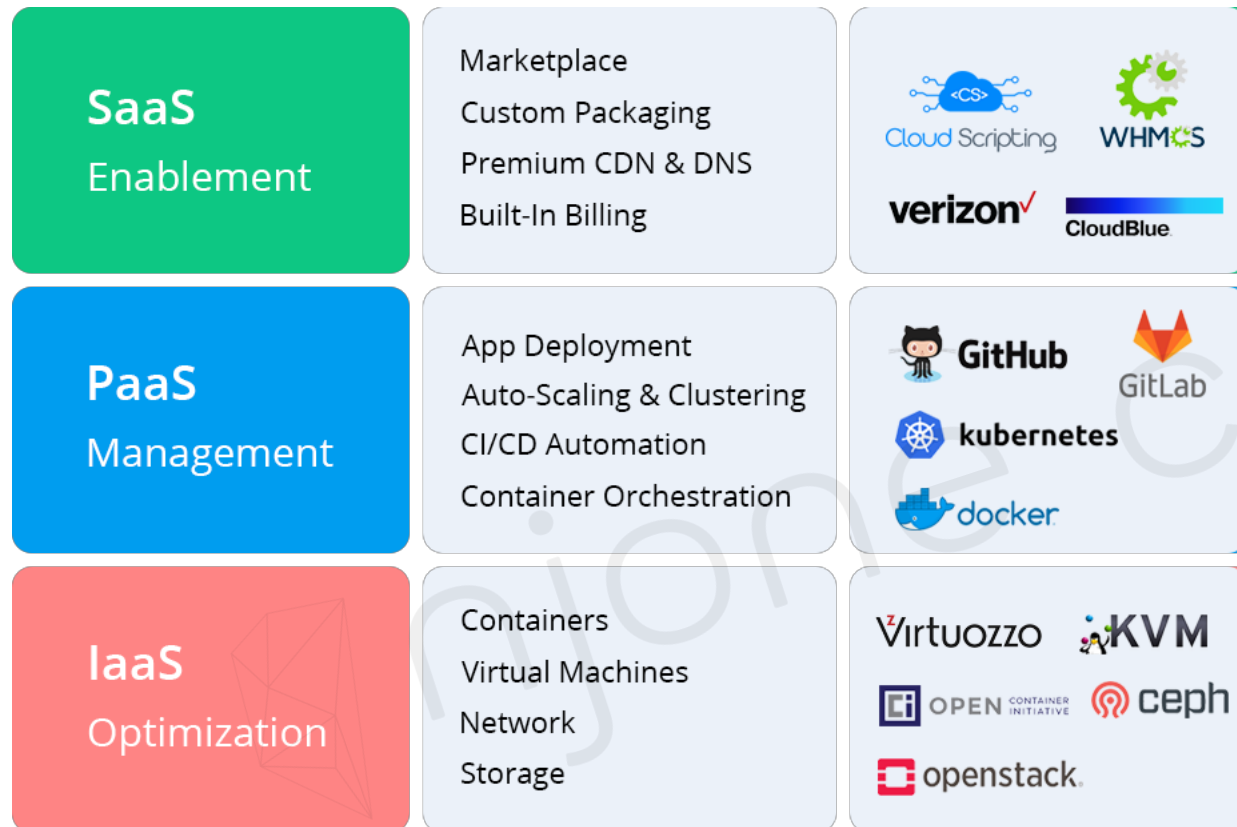


상용 클라우드에서의 애플리케이션 배포와 관리

anyone company

Cloud 환경에서의 서비스 배포

Cloud Service

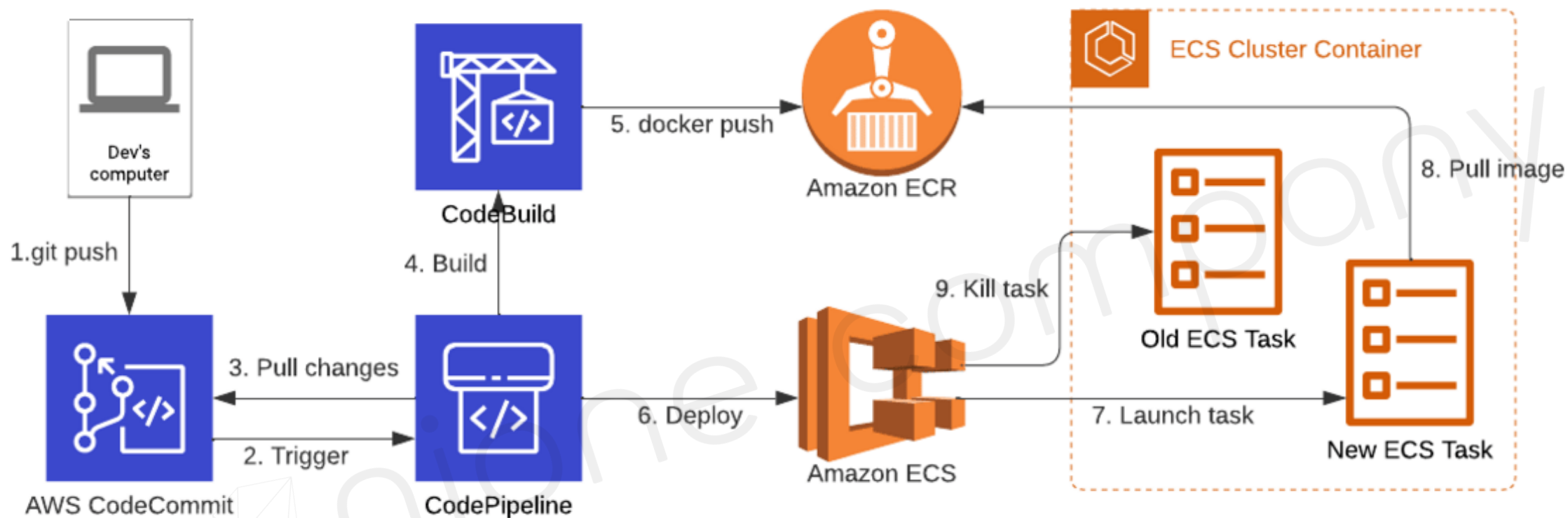


Microsoft Azure

NAVER
CLOUD PLATFORM

관리형 컨테이너 서비스 사용

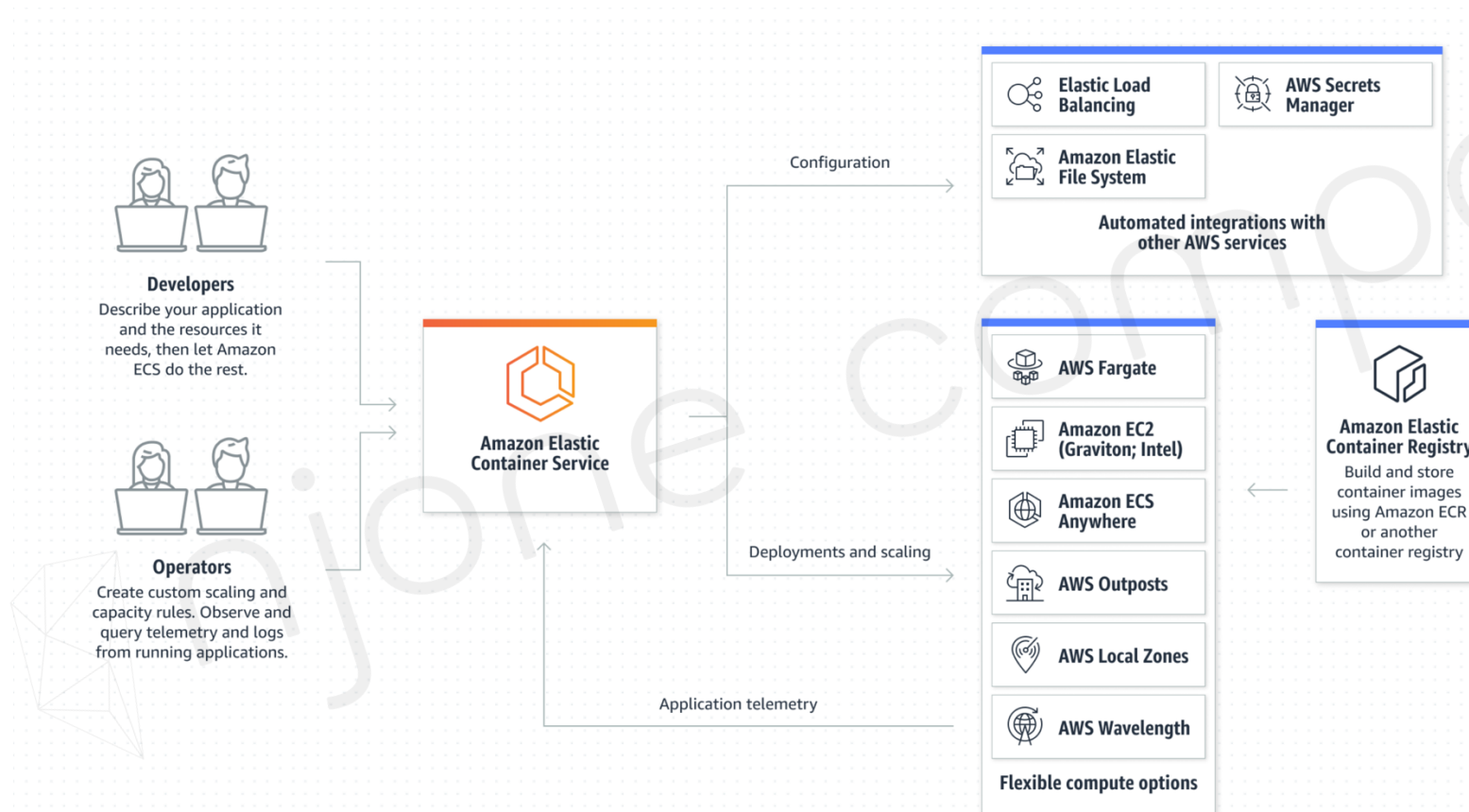
Cloud 환경의 관리형 컨테이너 서비스



Cloud 환경의 관리형 컨테이너 서비스

■ AWS ECS (Elastic Container Service) → Orchestration Tools

컨테이너의 생성과 종료, 자동 배치 및 복제, 로드 밸런싱, 클러스터링, 장애복구, 스케줄링



AWS ECR(Elastic Container Registry) 사용

- IAM 생성

- | AmazonEC2ContainerRegistryFullAccess

- ECR 생성

- | Private Repository

- Docker Image 빌드

- | Docker Client 인증

```
$ aws ecr get-login-password --region<REGION> | \
  docker login --username <USER> --password-stdin <AWS_ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com
```

- | Docker 이미지 빌드

```
$ docker build -t <IMAGE_NAME> .
```

AWS ECR 사용

- Docker Image Push

- | TAG 명 설정

```
$ docker tag <IMAGE_NAME>:<TAG> <NEW_IMAGE_NAME>:<NEW_TAG>
```

- | Docker Image Push

```
$ docker push <IMAGE_NAME> .<TAG>
```

- Docker Image Pull

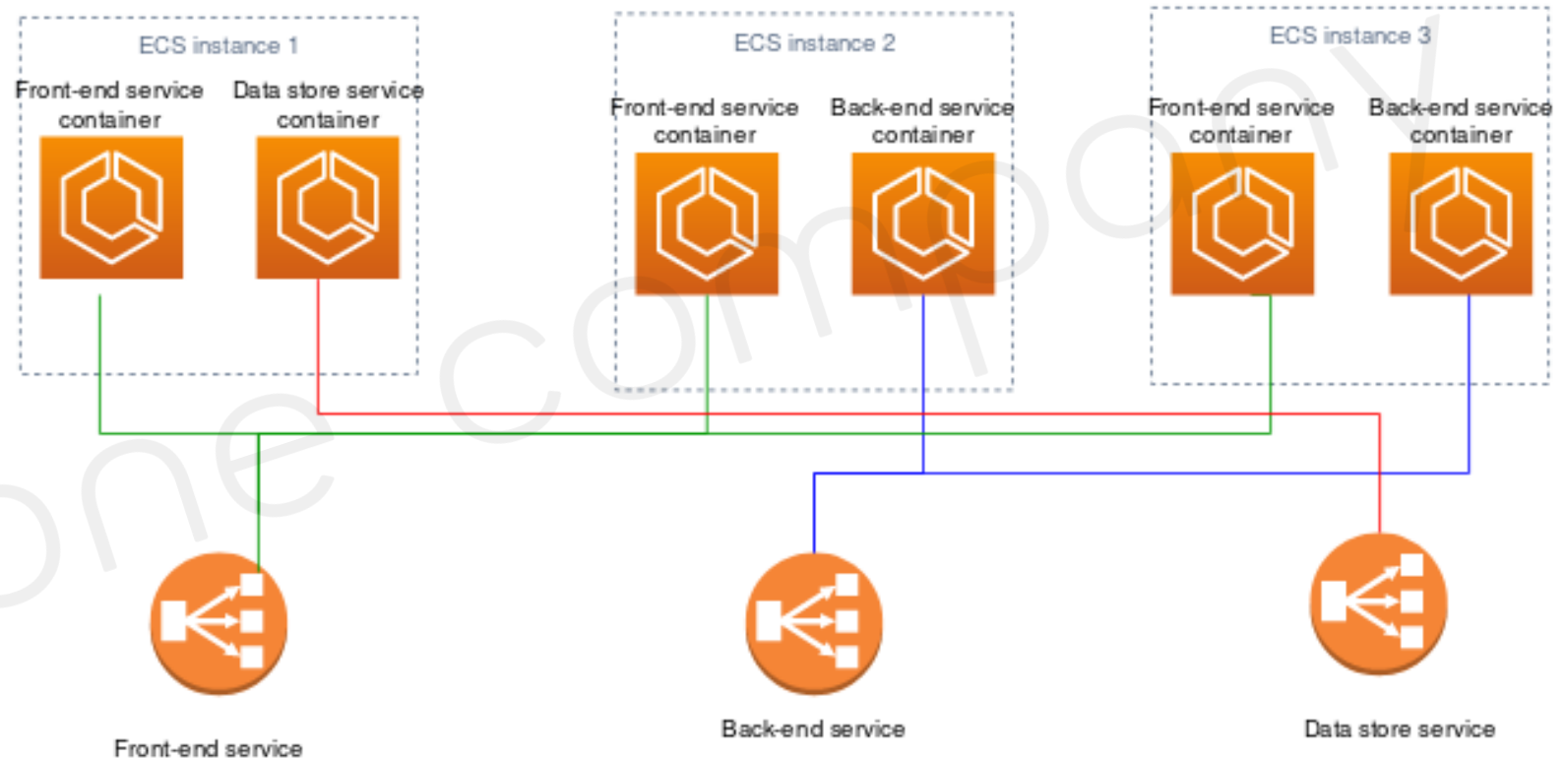
- | Docker Image Pull

```
$ docker pull <IMAGE_NAME>:<TAG>
```

AWS ECS 사용

■ Amazon Elastic Container Service

- | ECS Cluster 정의
- | ECS 작업 정의
- | ECS 작업 생성
- | ECS 서비스 생성



AWS ECS 사용

- 1. ECS 클러스터 생성

- | 클러스터 이름
- | AWS Fargate(서버리스)

- 2. ECS 작업 정의 (Task Definition)

- | 태스크 역할 → `ecsTaskExecutionRole`
- | 네트워크 모드 선택 → `bridge`(Docker 기본 가상 네트워크), `awsvpc` (fargate 사용 시 선택)
- | 작업에 할당 될 메모리와 CPU 총량 선택
- | 컨테이너 추가 → 포트 매핑

AWS ECS 사용

■ 3. ECS 작업 생성

- | 용량 공급자 → FARGATE
- | 애플리케이션 유형 → 태스크(작업)
- | 패밀리 → <2번에서 생성한 작업>

■ 4. ECS 서비스 생성

- | 용량 공급자 → FARGATE
- | 애플리케이션 유형 → 서비스
- | 패밀리 → <2번에서 생성한 작업>